

Loss Functions and ERM

What are we actually minimizing?

Topic 5 of Core ML Concepts

$$\arg \min_{\theta} \frac{1}{n} \sum_{i=1}^n \ell(f_{\theta}(x_i), y_i)$$

Zooming in on the symbol that tells a model what “better” means

True Risk vs. Empirical Risk

True risk · what we want

$$R(\theta) = \mathbb{E}_{(x,y) \sim \mathcal{D}}[\ell(f_{\theta}(x), y)]$$

Average loss over the unknown population distribution.

Empirical risk · what we have

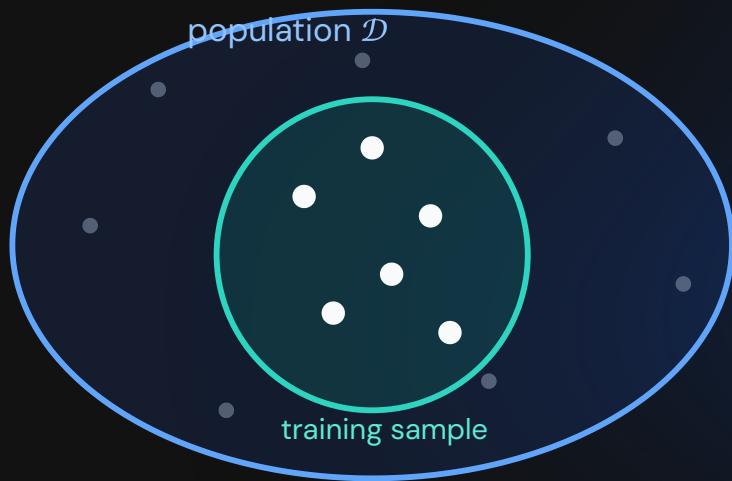
$$\hat{R}(\theta) = \frac{1}{n} \sum_{i=1}^n \ell(f_{\theta}(x_i), y_i)$$

Average loss over one finite training sample.

$$\hat{\theta}_{\text{ERM}} = \arg \min_{\theta} \hat{R}(\theta)$$

Empirical Risk Minimization = “fit the model to the training data”

The Generalization Gap



$R(\theta)$ low

The model fits observed examples.

$R(\theta)$ unknown

Future performance may still be poor.

Overfitting means $R(\theta)$ falls while the gap $R(\theta) - \hat{R}(\theta)$ grows.

What Makes a Useful Loss?



Aligned

low for predictions we value, high for costly mistakes



Optimizable

differentiable or subdifferentiable enough to supply direction



Well-shaped

convex when possible, avoiding unnecessary local minima



Task-aware

reflect target type, outliers, imbalance, and error costs

The easiest loss to optimize is not always the metric stakeholders care about—so use a defensible surrogate.

Regression: Squared vs. Absolute Error



Squared error · L2 loss

smooth; large misses dominate; Gaussian noise model

$$\ell(y, \hat{y}) = (y - \hat{y})^2$$

Absolute error · L1 loss

robust to outliers; corner at zero; Laplace noise model

$$\ell(y, \hat{y}) = |y - \hat{y}|$$

One Outlier, Two Very Different Objectives

```
import numpy as np

y_true = np.array([3., -.5, 2., 7., 50.])
y_pred = np.array([2.5, 0., 2., 8., 5.])

residuals = y_true - y_pred
squared = residuals ** 2
absolute = np.abs(residuals)

print(squared) # [0.25, 0.25, 0, 1, 2025]
print(absolute) # [0.5, 0.5, 0, 1, 45]
```

2025

squared contribution of the outlier

45

absolute contribution of the outlier

Classification: Why Not Optimize Accuracy?

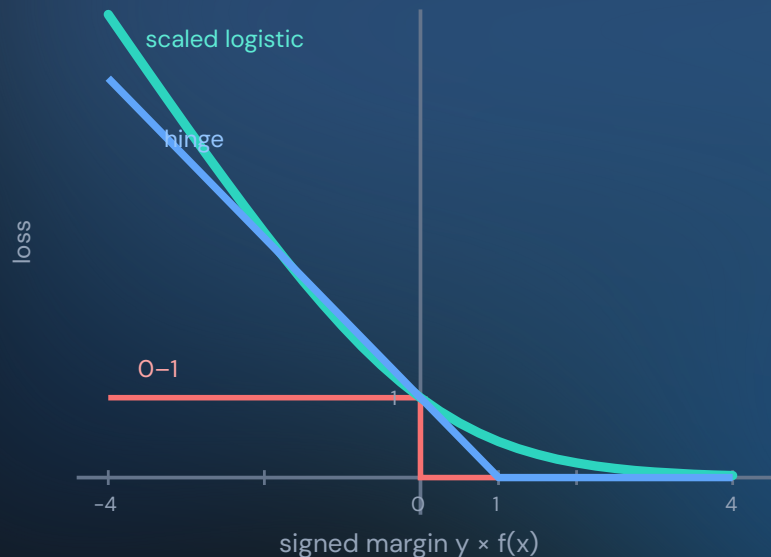


$$\ell_{0-1} = 1[\hat{y} \neq y]$$

- Flat almost everywhere
- Jumps at the decision boundary
- Gradient is zero or undefined
- Gives no signal about which direction improves the parameters

Train with a smooth **surrogate loss**; report accuracy when it suits the task.

Surrogate Losses Supply a Direction



Logistic loss

smooth; rewards calibrated probabilities
curve scaled by $1/\ln 2$ to upper-bound 0-1 loss

Hinge loss

penalizes points inside the margin; support vector machines

Both provide useful upper bounds on the abrupt 0-1 objective.

Where Log Loss Comes From: Bernoulli Likelihood

- Model a binary label $y \in \{0, 1\}$ as a coin flip with success probability $\hat{p} = f_{\theta}(x)$
- The Bernoulli probability mass function writes both cases in one formula: $P(y \mid \hat{p}) = \hat{p}^y (1 - \hat{p})^{1-y}$
- Assuming examples are drawn independently, the likelihood of the whole dataset is a product
- Maximizing likelihood is equivalent to minimizing its negative logarithm — logs turn products into sums

$$\begin{aligned}\mathcal{L}(\theta) &= \prod_{i=1}^n \hat{p}_i^{y_i} (1 - \hat{p}_i)^{1-y_i} \\ -\log \mathcal{L}(\theta) &= -\sum_{i=1}^n [y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i)] \\ \frac{1}{n} (-\log \mathcal{L}(\theta)) &= \frac{1}{n} \sum_{i=1}^n \ell_{\log}(y_i, \hat{p}_i)\end{aligned}$$

Cross-entropy loss is negative log-likelihood, averaged — ERM under log loss = maximum likelihood estimation.

Cross-Entropy Punishes Confident Mistakes

$$\ell_{\log}(y, \hat{p}) = -[y \log \hat{p} + (1 - y) \log(1 - \hat{p})]$$

$y = 1, \hat{p} = .99$

0.01

confident + correct

$y = 1, \hat{p} = .50$

0.69

uncertain

$y = 1, \hat{p} = .01$

4.61

confident + wrong

```
loss = -(y*np.log(p) + (1-y)*np.log(1-p))
```

In code, clip probabilities away from exactly 0 and 1 for numerical stability.

The Objective Used in Practice

$$\hat{\theta} = \arg \min_{\theta} \underbrace{\frac{1}{n} \sum_{i=1}^n \ell(f_{\theta}(x_i), y_i)}_{\text{empirical risk}} + \underbrace{\lambda R(\theta)}_{\text{regularization}}$$

Loss

defines each prediction's cost

Average

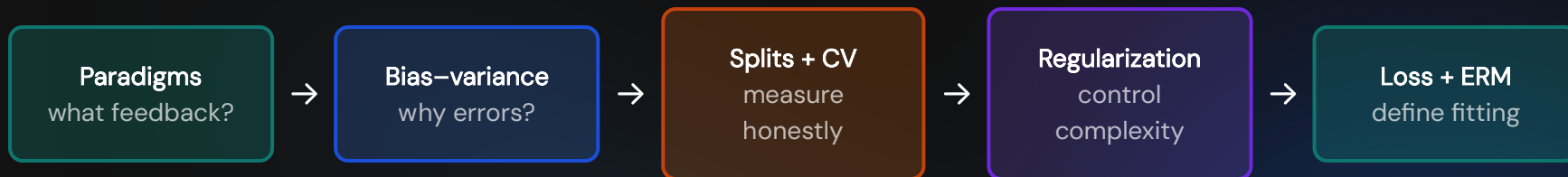
turns examples into empirical risk

Penalty

controls the generalization gap

Optimization minimizes the objective; validation checks whether that objective transfers to unseen data.

Core ML Concepts: One Connected Story



Next unit: **Supervised Learning — Regression**

We now have the vocabulary to build and evaluate concrete models.